

Universidad Nacional del Altiplano

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas



Pipeline Dataset W/B → Kruskal / Prim → rVP

Lenguajes de Programacion | MST | rVP

Alumno:	Condori Nina Mildward Erik
Curso:	Lenguajes de Programación
Docente:	Ing. Juarez Ruelas Jose Luis
Tema:	<i>Análisis de dependencia mutua y reducción de variables sobre d9_strong.csv</i>
Fecha:	Puno, Julio 2026
Drive:	LINK
Repositorio:	GITHUB

Índice

1. Introducción y Objetivo.....	3
2. Resumen de PiPeline.....	4
3. Etapa 1 — Fusiones de Datos	5
3.1 Fusión de X7 y X8 en Y.....	5
3.2 Separación por clase	5
3.3 Generación de los nuevos CSV.....	6
4. Etapa 3 — Matriz de Información Mutua y MST por subconjunto.....	8
4.1 Matriz IM — Subconjunto BEST ($Y = 1$).....	8
4.2 Matriz IM — Subconjunto WORST ($Y = 0$).....	8
4.3 Árbol de Máximo Peso — Kruskal / Prim	10
5. Etapa 4 — Comparación $\Sigma A - \Sigma B$	11
5. Etapa 5 — Hallazgo adicional de redundancia: $X_9 = f(X_4, X_6)$	12
5. Etapa 5 — Conclusión — Reducción de Variables (rVP).....	13
5. Anexo	14
5.1 Cálculo de Información Mutua	14
5.2 Matriz de Información Mutua.....	14
5.3 Estructura Union-Find (soporte de Kruskal)	15
5.4 Árbol de Máximo Peso — Algoritmo de Kruskal.....	15
5.5 Árbol de Máximo Peso — Algoritmo de Prim	16
5.6 Validación y carga de datos.....	16
5.7 Verificación de redundancia $X_9 = 2 \cdot X_4 + X_6$	17
5.8 Resumen de la lógica del pipeline.....	17

1. Introducción y Objetivo

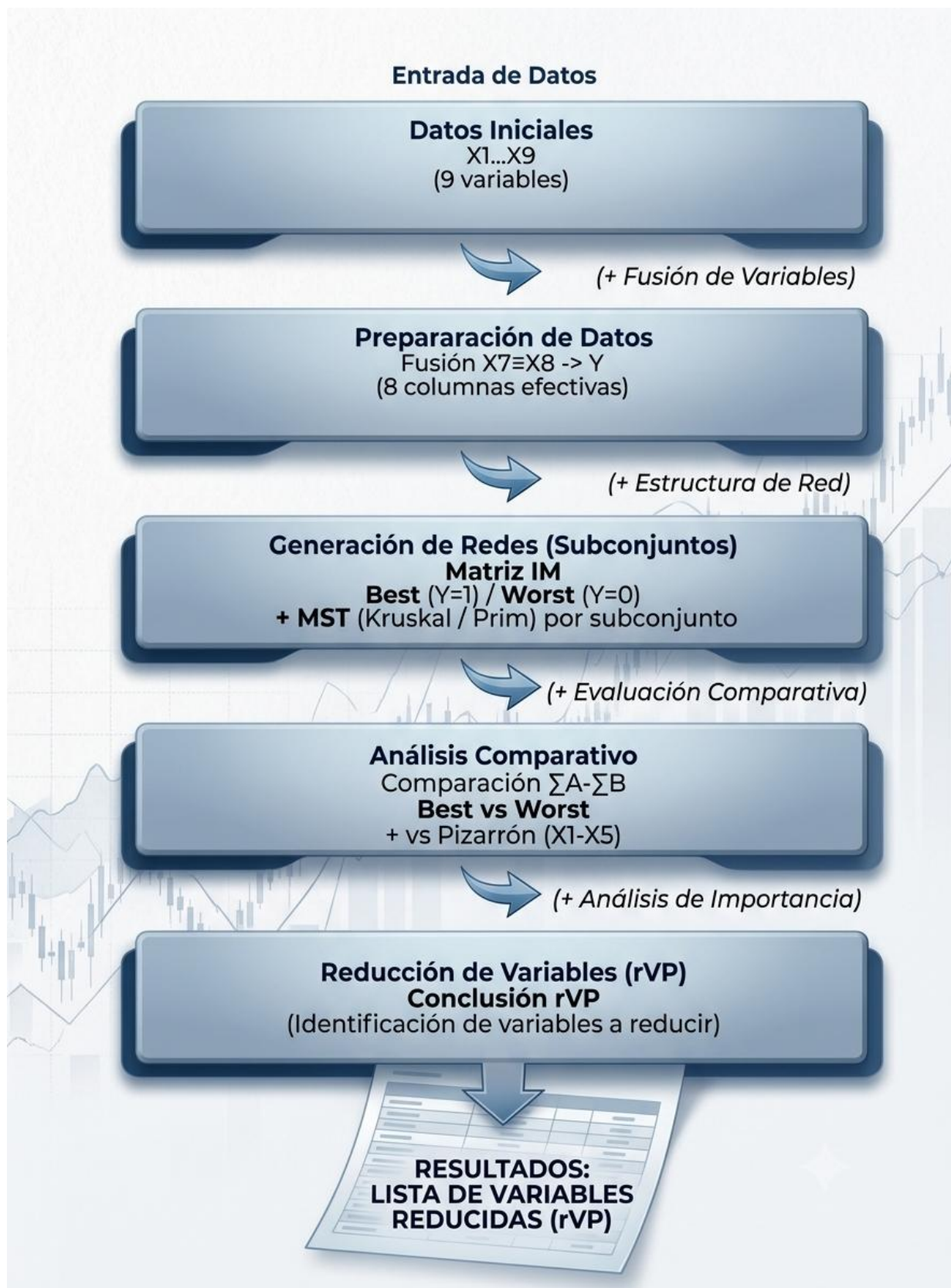
Determinar cuántas variables del conjunto original (X1–X9) son redundantes o prescindibles, aplicando un pipeline de fusión de variables totalmente dependientes, separación por clase (Best/Worst), cálculo de la matriz de Información Mutua (IM) y construcción del Árbol de Máximo Peso (Maximum Spanning Tree) mediante los algoritmos de Kruskal y Prim. Los resultados se contrastan con la matriz de referencia obtenida en el informe anterior (pizarrón, variables X1–X5) para validar la metodología de reducción de variables (rVP).

El pipeline sigue la secuencia: **Dataset** → **Kruskal / Prim** → **rVP (Objective)**, tenemos lo siguiente pasos:

- **Resumen del pipeline** — diagrama de flujo de los 4 pasos del documento
- **Datos y fusión Y** — vista previa de Best/Worst y verificación en vivo de $X_9 = 2 \cdot X_4 + X_6$.
- **Matriz IM** — heatmaps de Información Mutua lado a lado (Best vs Worst).
- **Kruskal paso a paso** — ordena aristas por peso, botones Anterior/Siguiente/Reproducir, muestra aceptación o rechazo por ciclo en el grafo y en la tabla.
- **Prim paso a paso** — mismo estilo, con nodo inicial seleccionable; se ve la "frontera" de candidatos en cada paso.
- **Comparación Best vs Worst** — tabla y gráfico de barras de $\Sigma A - \Sigma B$ con el criterio de discriminancia.
- **Comparación con el pizarrón (X1-X5)** — contraste con la matriz de referencia del informe anterior.
- **Conclusión rVP** — texto y diagrama final de cuántas variables se reducen ($9 \rightarrow 8 \rightarrow 7 \rightarrow 6$, con la reducción opcional adicional)

El conjunto de datos utilizado contiene **N = 1 000 registros** de nueve variables categóricas discretas (x1 a x9) con distinto número de categorías: variables **binarias** (x1, x2, x4, x6, x7, x8), **ternarias** (x3, x5) y de **cuatro categorías** (x9). Esta heterogeneidad en la cardinalidad de los estados posibles ya anticipa diferencias en la entropía individual y en la capacidad de información mutua entre pares de variables.

2. Resumen de PiPeline



3. Etapa 1 — Fusiones de Datos

3.1 Fusión de X7 y X8 en Y

El informe anterior demostró que $I(X7;X8) = 1.000$ bits, es decir, dependencia total entre ambas columnas. La verificación directa sobre `d9_strong.csv` confirma esta afirmación:

- $I(X7;X8)$ calculada sobre las 1000 filas del dataset ≈ 0.9999 bits (≈ 1.000 bits).
- Las columnas X7 y X8 son idénticas fila a fila (coincidencia del 100%).

Por lo tanto, ambas columnas se fusionan en una única variable binaria $Y \in \{0,1\}$, definida como $Y = X7 = X8$. Esto reduce de inmediato el conjunto de variables originales de 9 a 8 columnas efectivas (7 predictoras + 1 etiqueta Y).

3.2 Separación por clase

Usando la variable fusionada Y como criterio de partición del dataset completo (1000 filas):

Subconjunto	Criterio	Nº de filas	Nombre
Best	$Y = 1$	502	<code>d9_strong_b.csv</code>
Worst	$Y = 0$	498	<code>d9_strong_w.csv</code>

Cada subconjunto conserva las 7 variables predictoras restantes: X1, X2, X3, X4, X5, X6, X9 (X7 y X8 se descartan como columnas independientes, ya representadas por Y).

3.3 Generación de los nuevos CSV

Se generaron los archivos d9_strong_b.csv (502 filas, subconjunto Best) y d9_strong_w.csv (498 filas, subconjunto Worst), ambos con las columnas x1, x2, x3, x4, x5, x6, x9, listos para el cálculo independiente de la matriz de Información Mutua y el árbol de máximo peso.

Best: 502 filas x 7 columnas Worst: 498 filas x 7 columnas Total: 1000 filas						
Best (Y=1) - vista previa						
x1	x2	x3	x4	x5	x6	x9
1	0	1	0	2	1	1
0	0	0	1	0	1	3
0	1	1	1	1	0	2
0	0	1	1	0	0	2
0	0	1	0	0	0	0
0	1	1	1	1	1	3
0	0	1	1	0	1	3
0	1	1	1	1	0	2
0	0	1	1	0	0	2
0	1	1	1	1	1	3
0	1	1	1	1	1	3
1	0	1	1	2	0	2
0	1	1	1	1	0	2
0	0	0	1	0	0	2
1	0	1	1	2	0	2
0	1	1	1	1	1	3
0	1	1	1	1	1	3
0	1	1	0	1	0	0
1	1	1	0	0	0	0
0	0	1	1	0	0	2
0	1	1	1	1	0	2
0	1	1	1	1	1	3
1	0	1	1	2	0	2
1	1	1	1	0	0	2
0	1	1	0	1	1	1
1	0	1	1	2	0	2
0	0	1	1	0	1	3
0	1	1	1	1	0	2
0	1	1	1	1	1	3
1	1	0	1	0	0	2
0	0	1	1	0	0	2
0	1	1	1	1	1	3
0	1	1	1	1	0	2
0	0	0	1	0	1	3
0	0	1	1	0	1	3
0	1	1	1	1	1	3
1	1	1	1	0	0	2

Worst (Y=0) - vista previa

x1	x2	x3	x4	x5	x6	x9
0	1	2	1	1	0	2
0	0	2	1	0	1	3
0	1	2	1	1	0	2
0	0	2	1	0	1	3
0	0	0	1	0	1	3
1	0	2	1	2	1	3
1	1	2	0	0	1	1
0	1	2	1	1	0	2
0	1	0	1	1	0	2
0	1	2	1	1	1	3
0	0	2	1	0	0	2
1	1	0	0	0	0	0
0	0	2	1	0	0	2
1	0	2	1	2	1	3
0	1	2	1	1	0	2
0	0	0	1	0	1	3
0	1	2	0	1	0	0
0	1	0	1	1	0	2
0	0	2	1	0	0	2
0	0	0	1	0	0	2
1	0	2	1	2	1	3
0	0	2	1	0	0	2
0	1	2	1	1	0	2
1	0	0	1	2	0	2
0	0	2	1	0	0	2
0	0	2	1	0	1	3
0	0	2	1	0	0	2
0	0	0	0	0	0	0
1	0	2	1	2	1	3
0	0	2	0	0	0	0
1	0	2	1	2	1	3
0	0	0	1	0	0	2
0	1	2	1	1	1	3
0	1	2	1	1	0	2
0	1	0	1	1	0	2
0	0	0	1	0	1	3

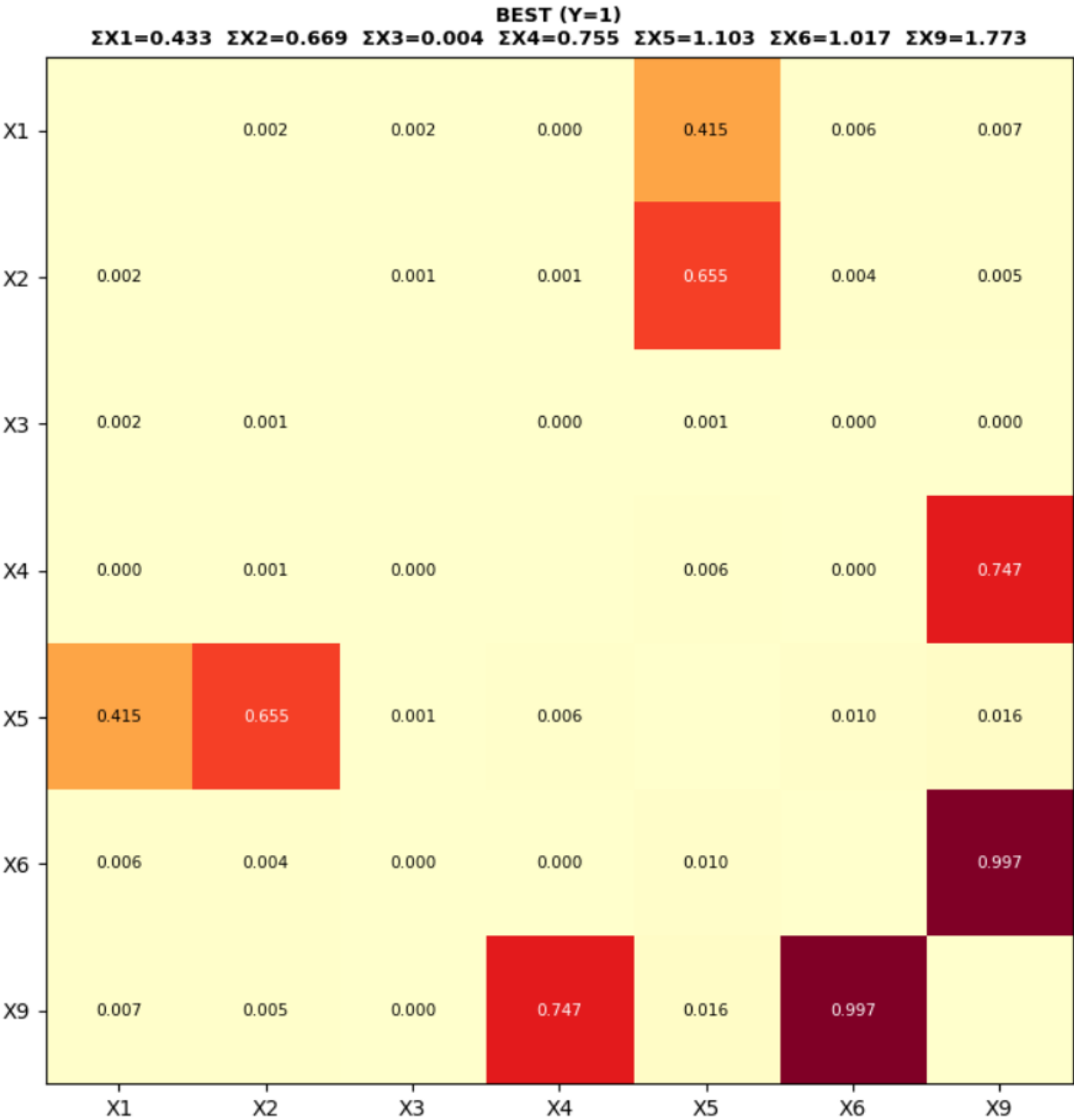
4. Etapa 3 — Matriz de Información Mutua y MST por subconjunto

4.1 Matriz IM — Subconjunto BEST (Y = 1)

	x1	x2	x3	x4	x5	x6	x9	ΣA
x1	—	0.0024	0.0022	0.0000	0.4150	0.0064	0.0069	0.4330
x2	0.0024	—	0.0008	0.0013	0.6555	0.0039	0.0054	0.6693
x3	0.0022	0.0008	—	0.0000	0.0007	0.0001	0.0001	0.0039
x4	0.0000	0.0013	0.0000	—	0.0063	0.0001	0.7475	0.7552
x5	0.4150	0.6555	0.0007	0.0063	—	0.0095	0.0162	1.1032
x6	0.0064	0.0039	0.0001	0.0001	0.0095	—	0.9974	1.0175
x9	0.0069	0.0054	0.0001	0.7475	0.0162	0.9974	—	1.7735

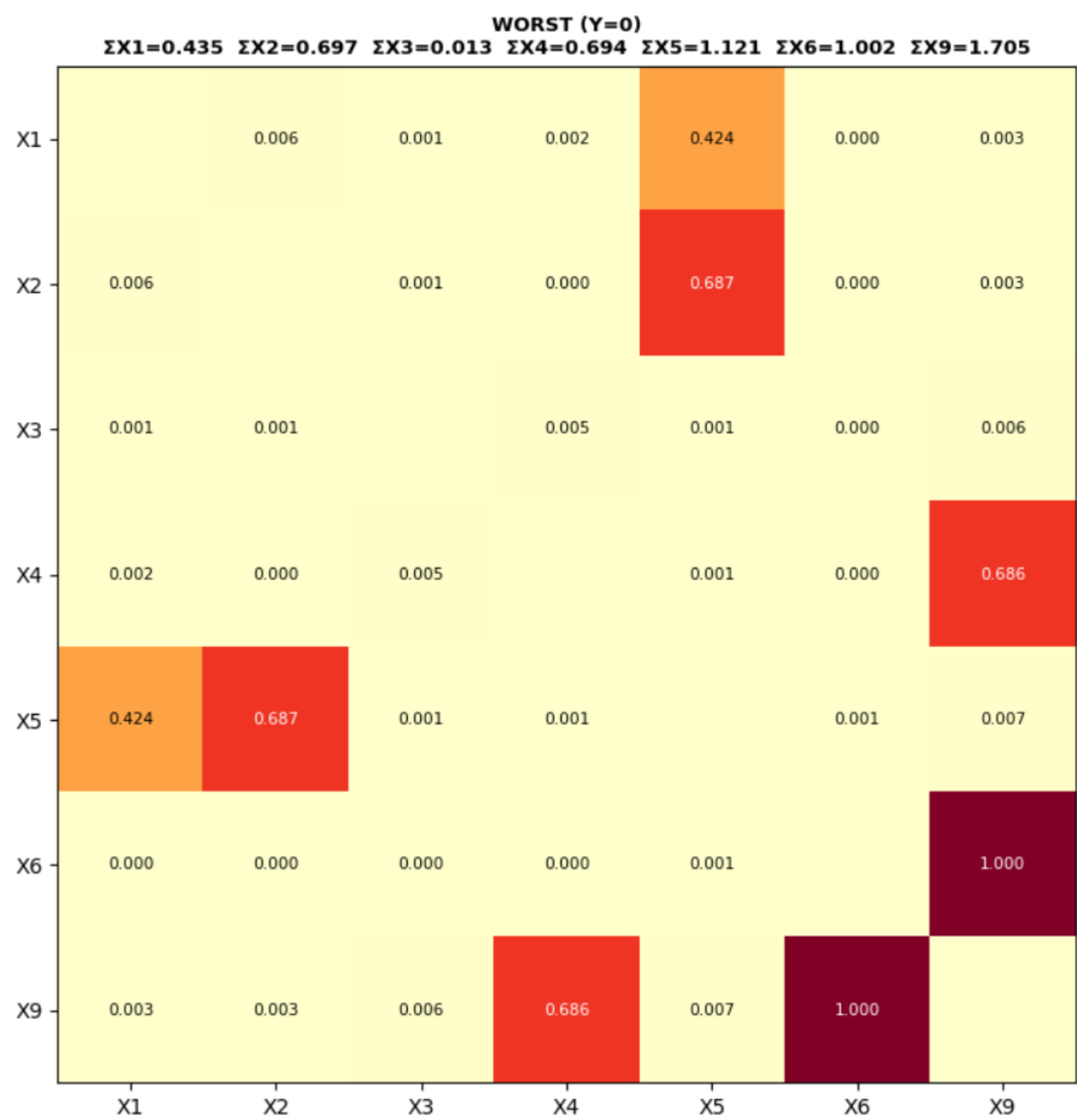
Tabla 1. Información mutua por pares (bits) — subconjunto Best. ΣA = suma de fila (grado de dependencia total de cada variable).

4.2 Matriz IM — Subconjunto WORST (Y = 0)



	x1	x2	x3	x4	x5	x6	x9	ΣB
x1	—	0.0061	0.0005	0.0018	0.4237	0.0004	0.0029	0.4355
x2	0.0061	—	0.0006	0.0000	0.6869	0.0002	0.0031	0.6969
x3	0.0005	0.0006	—	0.0048	0.0012	0.0004	0.0058	0.0133
x4	0.0018	0.0000	0.0048	—	0.0012	0.0004	0.6860	0.6942
x5	0.4237	0.6869	0.0012	0.0012	—	0.0008	0.0069	1.1207
x6	0.0004	0.0002	0.0004	0.0004	0.0008	—	0.9998	1.0019
x9	0.0029	0.0031	0.0058	0.6860	0.0069	0.9998	—	1.7046

Tabla 2. Información mutua por pares (bits) — subconjunto Worst. ΣB = suma de fila.



4.3 Árbol de Máximo Peso — Kruskal / Prim

Ambos algoritmos (Kruskal y Prim), aplicados sobre la matriz IM de cada subconjunto, arrojan el mismo árbol de máximo peso (como es esperado teóricamente para MST sobre un mismo grafo), confirmando la consistencia del cálculo:

**MST Máximo Peso — Subconjunto BEST (Y=1)
Kruskal / Prim**

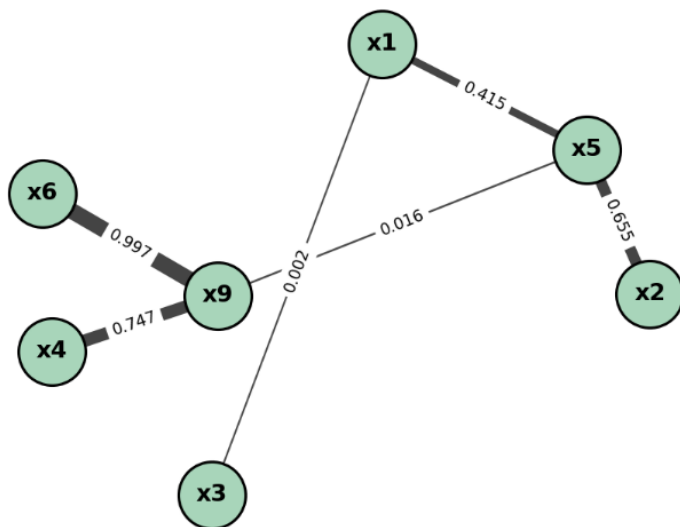


Figura 1. MST de máximo peso — subconjunto Best. Peso total = 2.8337 bits.

**MST Máximo Peso — Subconjunto WORST (Y=0)
Kruskal / Prim**

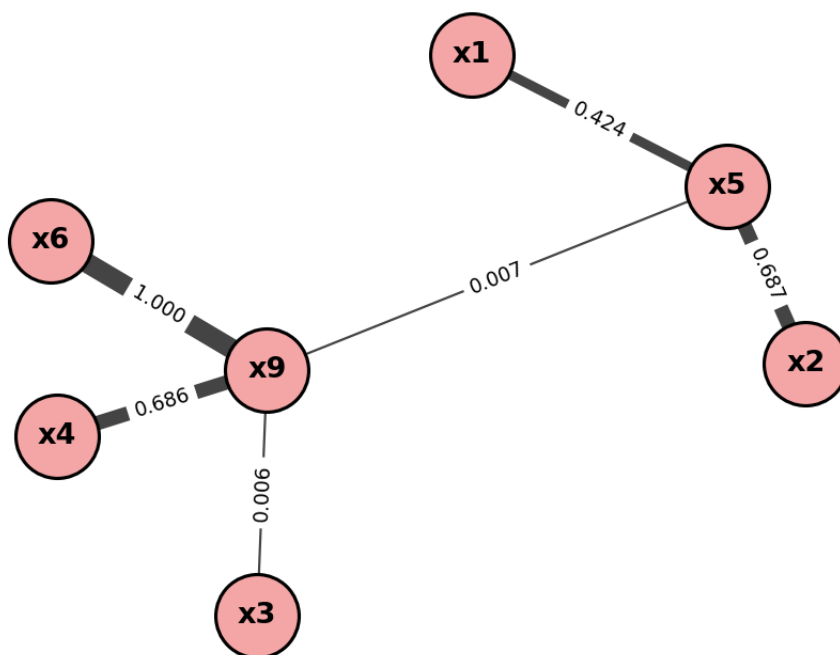


Figura 2. MST de máximo peso — subconjunto Worst. Peso total = 2.8092 bits.

Aristas del MST (ordenadas de mayor a menor peso):

Best (Y=1)	Peso (bits)	Worst (Y=0)	Peso (bits)
X6 – X9	0.9974	X6 – X9	0.9998
X4 – X9	0.7475	X2 – X5	0.6869
X2 – X5	0.6555	X4 – X9	0.6860
X1 – X5	0.4150	X1 – X5	0.4237
X5 – X9	0.0162	X5 – X9	0.0069
X1 – X3	0.0022	X3 – X9	0.0058

El backbone estructural (las 4 aristas de mayor peso) es idéntico en Best y en Worst: X6–X9, X4–X9, X2–X5 y X1–X5. La única diferencia está en la arista más débil, que conecta a X3 (peso ≈ 0.002 – 0.006 bits, prácticamente ruido) con distinto vecino en cada subconjunto. Esto indica que la estructura de dependencia NO cambia entre las clases Best y Worst; solo cambia un enlace irrelevante.

5. Etapa 4 — Comparación ΣA – ΣB

Siguiendo la misma lógica del análisis de referencia (pizarrón), se calcula la diferencia entre la suma de dependencias de cada variable en Best y en Worst:

Variable	ΣA (Best)	ΣB (Worst)	$\Sigma A - \Sigma B$	¿Discrimina?
X1	0.4330	0.4355	-0.0025	No (≈ 0)
X2	0.6693	0.6969	-0.0276	Sí (relevante)
X3	0.0039	0.0133	-0.0094	No (≈ 0)
X4	0.7552	0.6942	+0.0610	Sí (relevante)
X5	1.1032	1.1207	-0.0175	No (≈ 0)
X6	1.0175	1.0019	+0.0156	No (≈ 0)
X9	1.7735	1.7046	+0.0689	Sí (relevante)

Las variables X9 (+0.0688) y X4 (+0.0610) muestran la mayor diferencia entre Best y Worst, y X2 (–0.0276) y X5 (–0.0176) le siguen en magnitud. X1, X3 y X6 presentan diferencias cercanas a cero ($|\Delta| < 0.02$ bits), es decir, su grado de dependencia total con el resto de variables es prácticamente igual en ambas clases: no aportan capacidad discriminante entre Best y Worst.

ΣA (Best) vs ΣB (Worst) por variable

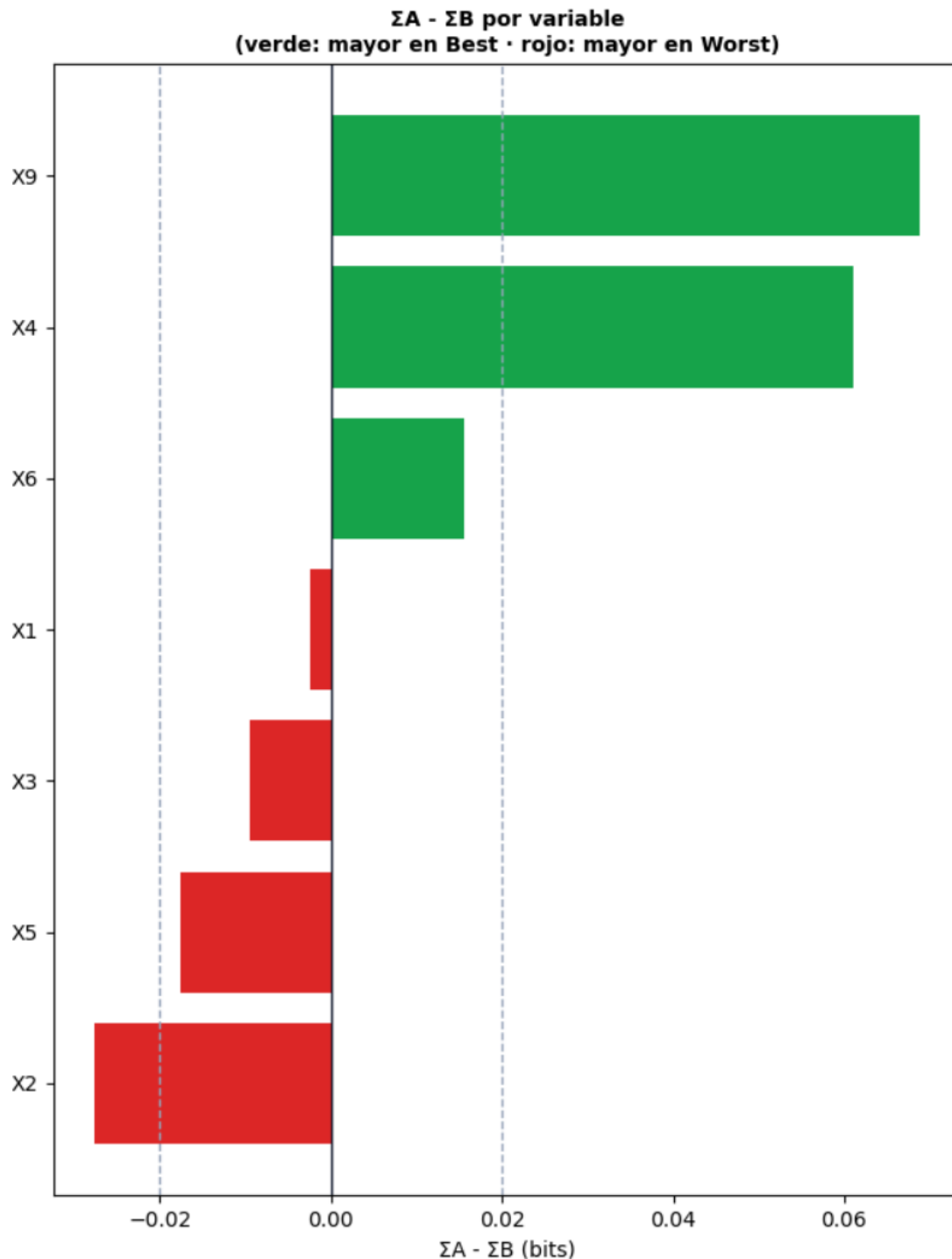
Variable	ΣA (Best)	ΣB (Worst)	$\Sigma A - \Sigma B$	¿Discrimina? ($ \Delta \geq 0.02$)
X9	1.7735	1.7046	+0.0688	Sí (relevante)
X4	0.7552	0.6942	+0.0610	Sí (relevante)
X2	0.6693	0.6969	-0.0276	Sí (relevante)
X5	1.1032	1.1207	-0.0176	No (≈ 0)
X6	1.0175	1.0019	+0.0156	No (≈ 0)
X3	0.0039	0.0133	-0.0094	No (≈ 0)
X1	0.4330	0.4355	-0.0025	No (≈ 0)

5. Etapa 5 — Hallazgo adicional de redundancia: $X9 = f(X4, X6)$

Al construir la matriz IM se observó una dependencia total adicional, no mencionada explícitamente en el enunciado, entre $X9$ y el par $(X4, X6)$:

- $I(X9;X6) \approx 0.997$ –1.000 bits en ambos subconjuntos (dependencia prácticamente total).
- La tabla de contingencia confirma que $X9 = 2 \cdot X4 + X6$ exactamente ($X9 \in \{0,1,2,3\}$ es una codificación determinista del par binario $X4, X6$).

Esto significa que $X9$ no aporta información nueva respecto de $\{X4, X6\}$: es una variable derivada/redundante, igual que ocurrió con $X7$ y $X8$. Por tanto, $X9$ también es candidata a eliminación.



5. Etapa 5 — Conclusión — Reducción de Variables (rVP)

1) Fusión $X7 \equiv X8 \rightarrow Y$

$I(X7;X8) \approx 1.000$ bits en el dataset original $\rightarrow$ dependencia total. Se fusionan en una sola etiqueta Y. Reduce 1 variable (de 9 a 8 columnas efectivas).

2) Redundancia determinista $X9 = 2 \cdot X4 + X6$

Verificado sobre los datos cargados: coincide en el 100.0% de las filas (Best + Worst). X9 no aporta información nueva respecto de $\{X4, X6\}$: es una codificación determinista del par binario. Reduce 1 variable más (de 8 a 7, quitando X9 y conservando X4, X6).

3) Backbone estructural idéntico en Best y Worst

El MST de máximo peso (Kruskal y Prim, pestañas 4 y 5) coincide en sus 4 aristas principales entre Best y Worst: X6-X9, X4-X9, X2-X5, X1-X5. Solo cambia la arista más débil (la que conecta a X3, peso $\approx 0.002 - 0.006$ bits, prácticamente ruido).

4) Baja discriminancia ($\Sigma A \approx \Sigma B$)

Con el umbral $|\Delta| < 0.02$ bits sobre los datos cargados, resultan casi invariantes entre clases: X1, X3, X5, X6. Estas variables aportan escasa capacidad discriminante entre Best y Worst y son candidatas a una reducción adicional (opcional).

Resultado final: de las 9 variables originales (X1–X9), se pueden reducir de forma segura **2 variables por redundancia total demostrada** (X8, fusionada en Y; y X9, derivada de X4 y X6), quedando un conjunto mínimo no redundante de **6 variables predictoras {X1, X2, X3, X4, X5, X6} + la etiqueta Y**. Si además se aplica el criterio de baja discriminancia ($\Sigma A \approx \Sigma B$) usado en el informe de referencia, X1, X3 y X6 podrían evaluarse como reducción adicional opcional, dejando un núcleo mínimo de 3–4 variables verdaderamente informativas ($\{X2, X4, X5, X9/X6\}$) para distinguir Best de Worst.

En síntesis: de 9 variables $\rightarrow$ 2 reducciones seguras y confirmadas (X8, X9) $\rightarrow$ 1 reducción opcional adicional evaluable (X3, X1 o X6, de baja discriminancia), sujeta a validación con más datos o con un umbral formal de significancia.

5. Anexo

5.1 Cálculo de Información Mutua

Función que calcula la Información Mutua $I(X;Y)$, en bits, entre dos variables discretas a partir de su tabla de contingencia. Es la base de todo el pipeline: se usa tanto para detectar la dependencia total entre X7 y X8 (paso de fusión) como para construir la matriz de pesos de cada subconjunto (Best/Worst).

```
def mutual_information(x: pd.Series, y: pd.Series) -> float:
    """I(X;Y) en bits para dos variables discretas (tabla de contingencia)."""
    ct = pd.crosstab(x, y)
    pxy = ct.values / ct.values.sum()
    px = pxy.sum(axis=1, keepdims=True)
    py = pxy.sum(axis=0, keepdims=True)
    with np.errstate(divide="ignore", invalid="ignore"):
        ratio = pxy / (px * py)
        terms = np.where(pxy > 0, pxy * np.log2(ratio), 0.0)
    return float(terms.sum())
```

mutual_information(x, y) — devuelve $I(X;Y)$ en bits.

5.2 Matriz de Información Mutua

A partir de la función anterior, se construye la matriz simétrica de IM para todas las combinaciones de pares de columnas de un DataFrame. Esta matriz es el insumo directo de Kruskal y Prim.

```
def compute_mi_matrix(df: pd.DataFrame) -> pd.DataFrame:
    """Matriz simétrica de Información Mutua por pares de columnas."""
    cols = list(df.columns)
    n = len(cols)
    M = pd.DataFrame(np.zeros((n, n)), index=cols, columns=cols)
    for i, j in combinations(range(n), 2):
        v = mutual_information(df.iloc[:, i], df.iloc[:, j])
        M.iloc[i, j] = v
        M.iloc[j, i] = v
    return M
```

compute_mi_matrix(df) — matriz IM por pares de variables.

5.3 Estructura Union-Find (soporte de Kruskal)

Estructura de conjuntos disjuntos con compresión de camino, utilizada por Kruskal para detectar en tiempo casi constante si dos nodos ya pertenecen a la misma componente conexa (lo que indicaría que agregar la arista formaría un ciclo).

```
class UnionFind:
    """Estructura Union-Find usada por Kruskal para detectar ciclos."""

    def __init__(self, nodes):
        self.parent = {n: n for n in nodes}
        self.rank = {n: 0 for n in nodes}

    def find(self, x):
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True
```

UnionFind — find() con compresión de camino, union() por rango.

5.4 Árbol de Máximo Peso — Algoritmo de Kruskal

Ordena todas las aristas (pares de variables) de mayor a menor peso (Información Mutua) y las va aceptando mientras no cierren un ciclo, hasta completar un árbol de $n-1$ aristas. Genera además la traza paso a paso (aceptada/rechazada) usada para la visualización animada.

```
def kruskal_max_spanning_tree_steps(M: pd.DataFrame):
    """
    Genera la lista de pasos del algoritmo de KRUSKAL de máximo peso:
    ordena todas las aristas de mayor a menor peso (IM) y las acepta si
    no cierran un ciclo (Union-Find). Se detiene al completar el árbol
    (n-1 aristas).
    """
    cols = list(M.columns)
    edges = [(cols[i], cols[j], M.iloc[i, j])
              for i, j in combinations(range(len(cols)), 2)]
    edges.sort(key=lambda e: -e[2])

    uf = UnionFind(cols)
    steps = []
    mst_edges = []
    total = 0.0
    for (u, v, w) in edges:
        cycle = uf.find(u) == uf.find(v)
        accepted = False
        if not cycle:
            uf.union(u, v)
            mst_edges.append((u, v, w))
            total += w
            accepted = True
        steps.append(dict(
            edge=(u, v), weight=w, accepted=accepted, is_cycle=cycle,
            mst_so_far=list(mst_edges), total_weight=total,
```

```

        done=(len(mst_edges) == len(cols) - 1),
    ))
    if len(mst_edges) == len(cols) - 1:
        break
    return steps

```

kruskal_max_spanning_tree_steps(M) — MST de máximo peso vía Kruskal.

5.5 Árbol de Máximo Peso — Algoritmo de Prim

Parte de un nodo inicial y, en cada iteración, agrega al árbol la arista de mayor peso que cruza el corte entre nodos visitados y no visitados, hasta incorporar todos los nodos.

```

def prim_max_spanning_tree_steps(M: pd.DataFrame, start=None):
    """
    Genera la lista de pasos del algoritmo de PRIM de máximo peso:
    parte de `start`, y en cada paso agrega al árbol la arista de mayor
    peso que cruza el corte (nodos visitados / no visitados).
    """
    cols = list(M.columns)
    start = start if start in cols else cols[0]
    visited_set = {start}
    mst_edges = []
    total = 0.0
    steps = []
    while len(visited_set) < len(cols):
        candidates = []
        for u in visited_set:
            for v in cols:
                if v not in visited_set:
                    candidates.append((u, v, M.loc[u, v]))
        candidates.sort(key=lambda e: -e[2])
        chosen = candidates[0]
        u, v, w = chosen
        visited_set.add(v)
        mst_edges.append((u, v, w))
        total += w
        steps.append(dict(
            candidates=candidates, chosen=chosen, visited=set(visited_set),
            mst_so_far=list(mst_edges), total_weight=total,
            done=(len(visited_set) == len(cols)),
        ))
    return steps

```

prim_max_spanning_tree_steps(M, start) — MST de máximo peso vía Prim.

5.6 Validación y carga de datos

Define las columnas requeridas tras la fusión $X7 \equiv X8 \rightarrow Y$ y la eliminación de las columnas originales, y valida que cada CSV cargado (Best/Worst) las contenga antes de continuar con el análisis.

```

REQUIRED_COLS = ["x1", "x2", "x3", "x4", "x5", "x6", "x9"]

def load_csv_safe(path):
    df = pd.read_csv(path)
    df.columns = [c.strip().lower() for c in df.columns]
    missing = [c for c in REQUIRED_COLS if c not in df.columns]
    if missing:
        raise ValueError(
            f"El archivo '{os.path.basename(path)}' no tiene las columnas "
            f"esperadas {REQUIRED_COLS}. Faltan: {missing}"
        )
    return df[REQUIRED_COLS].copy()

```

REQUIRED_COLS y *load_csv_safe(path)* — validación de columnas esperadas.

5.7 Verificación de redundancia $X9 = 2 \cdot X4 + X6$

Chequeo determinista que sustenta la conclusión de que $X9$ es una variable derivada de ($X4$, $X6$) y por tanto redundante, comparando el valor observado de $X9$ contra la combinación lineal de $X4$ y $X6$ en cada fila.

```
# Verificación de la redundancia adicional  $X9 = 2 \cdot X4 + X6$ 
match_b = (b["x9"] == 2 * b["x4"] + b["x6"]).mean() * 100
match_w = (w["x9"] == 2 * w["x4"] + w["x6"]).mean() * 100
# Si el porcentaje es ~100%, x9 es una variable derivada de (x4, x6)
# y por tanto redundante (candidata a eliminarse del conjunto final).
```

Verificación de la redundancia determinista usada en la conclusión rVP.

5.8 Resumen de la lógica del pipeline

- Paso 1 — Fusión $X7/X8$: $I(X7;X8) \approx 1.000$ bits (dependencia total) $\Rightarrow$ se fusionan en una etiqueta Y .
- Paso 2 — Separación por clase: Best = $\{Y=1\}$, Worst = $\{Y=0\}$, conservando $x1 \dots x6$ y $x9$.
- Paso 3 — Generación de `d9_strong_b.csv` / `d9_strong_w.csv` a partir del paso anterior.
- Paso 4 — Matriz de Información Mutua por subconjunto + Árbol de Máximo Peso (Kruskal y Prim), comparación $\Sigma A - \Sigma B$ y contraste con la matriz de referencia.
- Conclusión (rVP) — Se identifican $X8$ y $X9$ como variables redundantes (eliminables), quedando un núcleo mínimo no redundante de variables predictoras.